SCSE3040 Machine Learning Operations — Practical 08

1. Student details

Name	Shaurya Nigam
Roll number	S24CSEU0497
Batch	89
Date of laboratory session	28/10/26

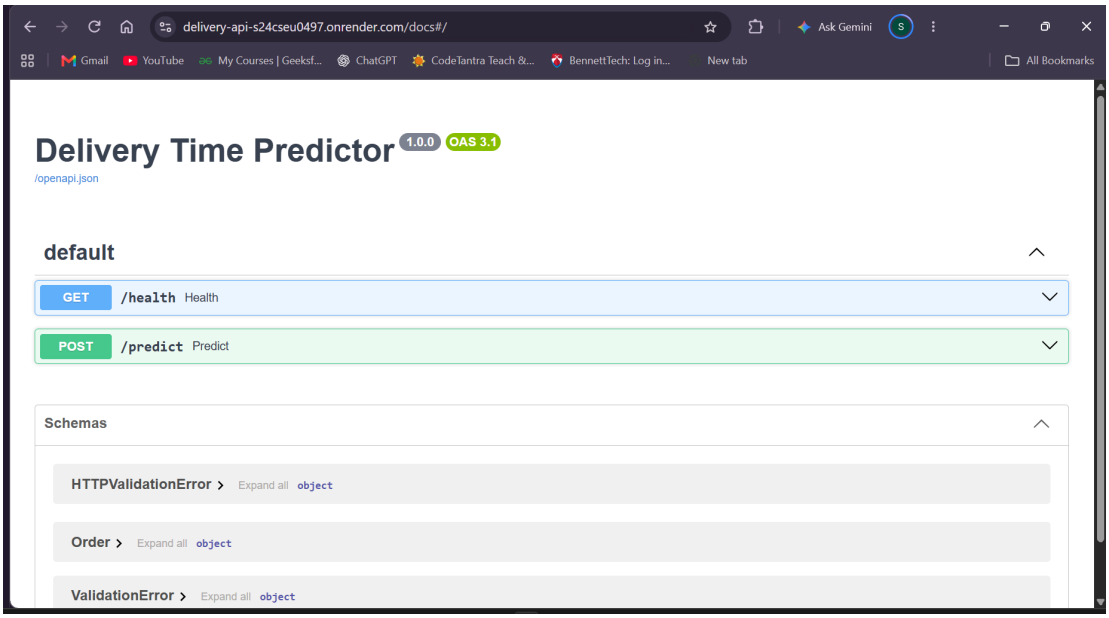
2. Repository link

https://github.com/shaauryaa/aiml-lab-assignments/tree/main/MLOps-SCSE3040/P08_docker

3. Public URL of the deployed service

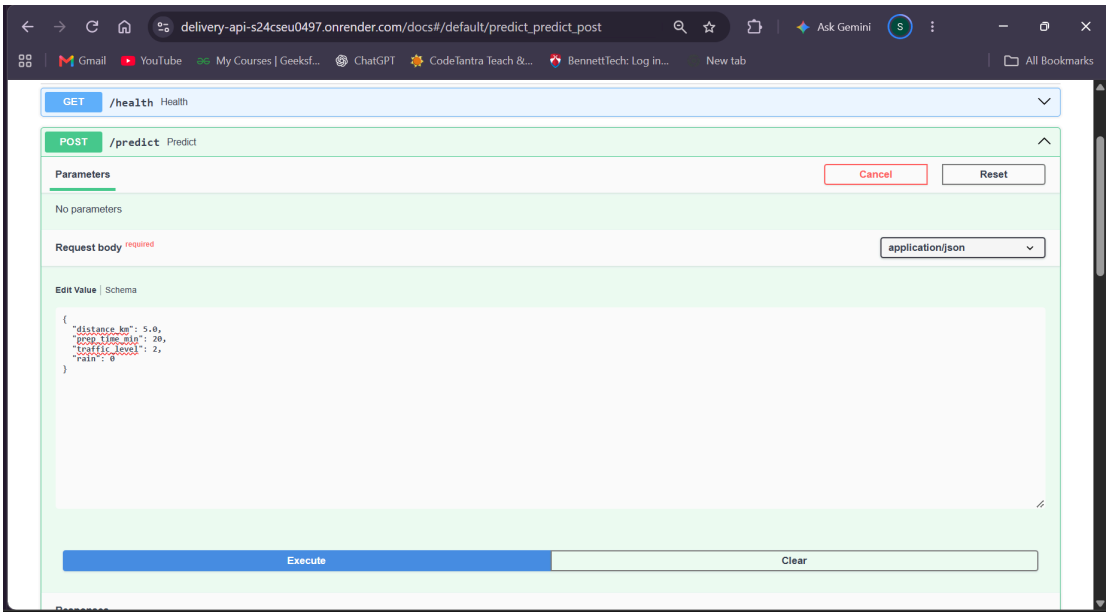
https://delivery-api-s24cseu0497.onrender.com

4. Interactive documentation page (`/docs`) showing `/predict`

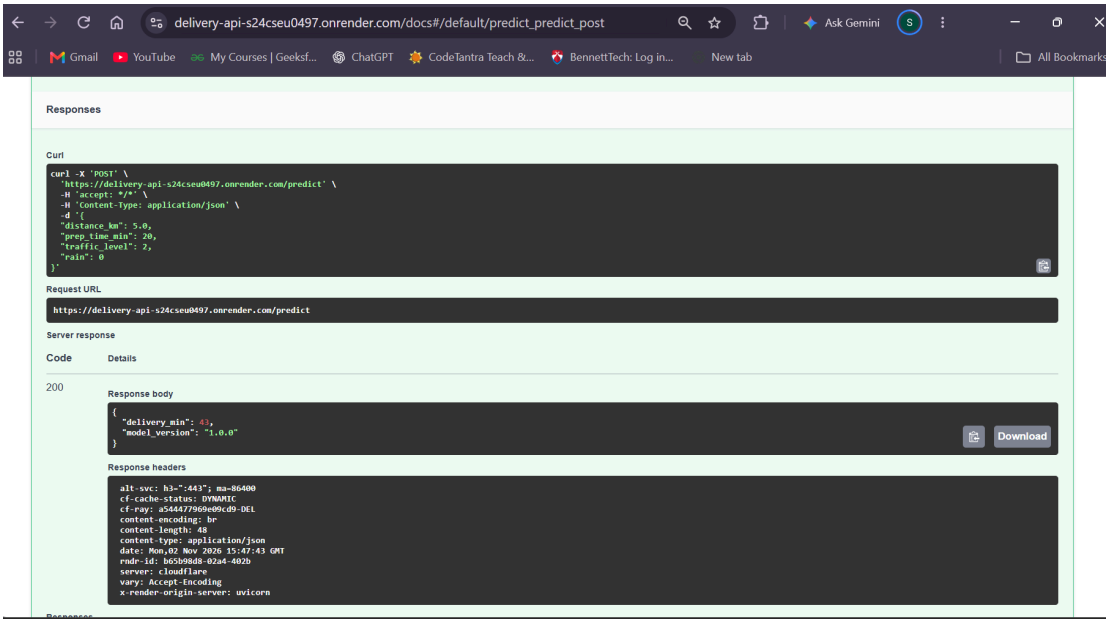


5. Successful call to `/predict` on the public URL

Request body sent:

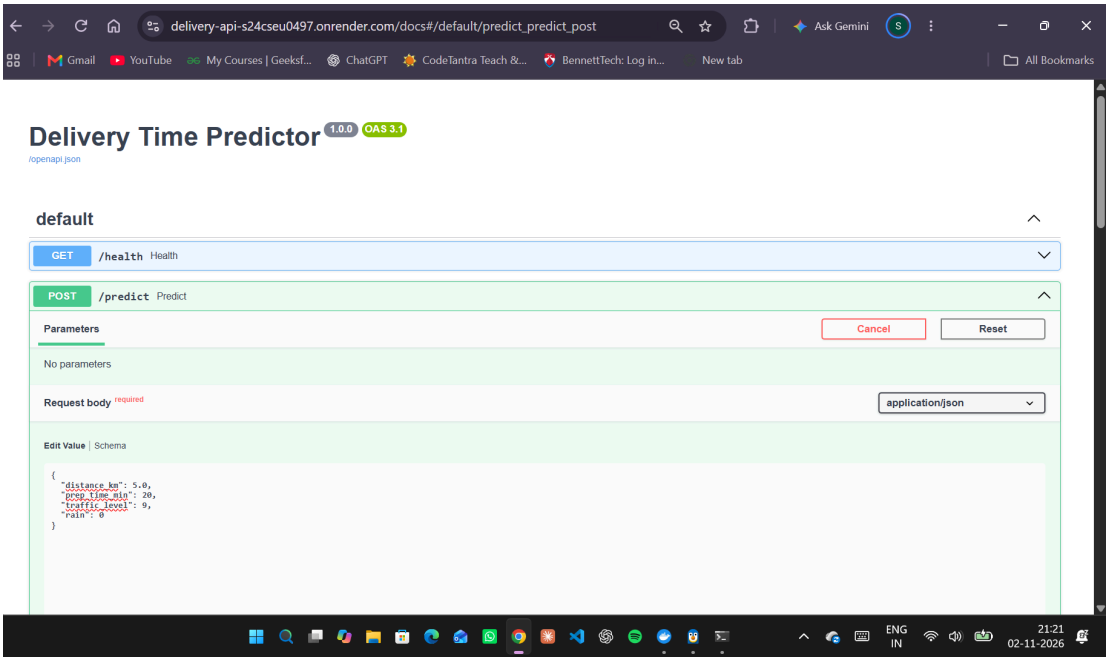


Prediction returned, code 200, `delivery_min` 43:

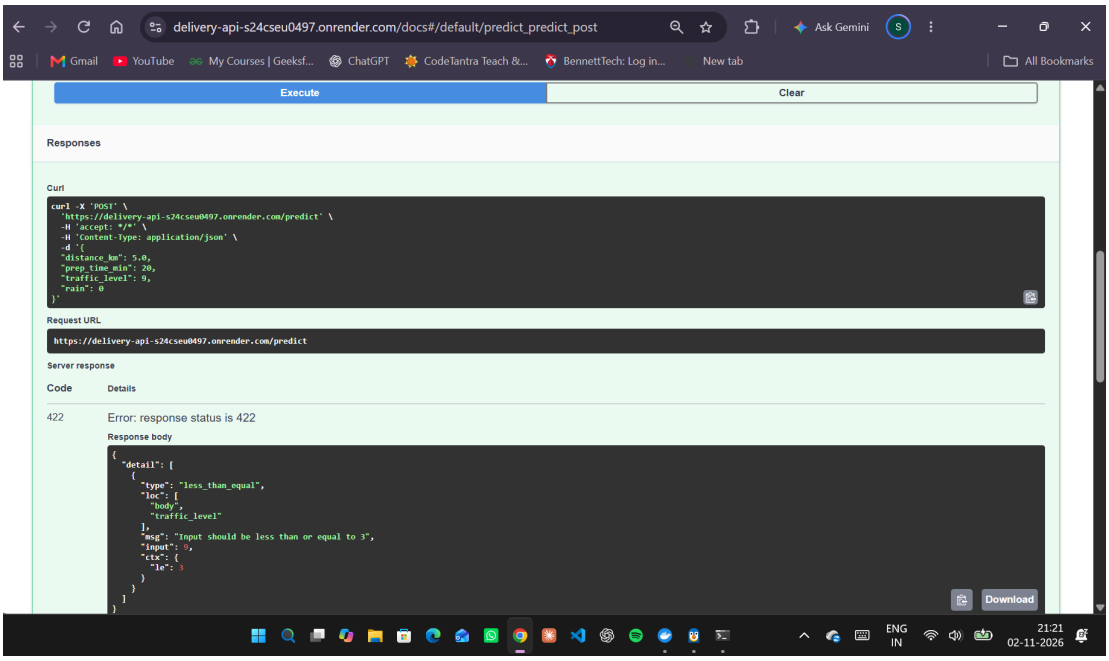


6. Rejected call: request validation refused bad input

Request body sent, with `traffic_level` set to 9 (the limit is 3):



Error returned, code 422:



7. Cloud platform

Render (free Web Service, built from the Dockerfile in the repository)

Practical 08 --- Packing the Service into a Container

A Dockerfile, an image, and a service that runs the same everywhere

SCSE3040 Machine Learning Operations · Bennett University · Session 2026-27

Lectures this follows	L13-L14
Course Outcome	CO3
Lab duration	120 minutes
Memory needed	about 900 MB (fine on a 4 GB or 8 GB machine)
Extra software	Docker Desktop
Marks	10

Aim

1. Explain what a container gives you that a virtual environment cannot.
2. Write a Dockerfile for the delivery-time service.
3. Build an image and run it, then call the service inside it.
4. Read a container's logs and shut it down cleanly.

What you need to know first

In P01 you pinned your library versions so a colleague could rebuild your environment. That solves half the problem. It does not pin the Python version, the operating system, the system libraries underneath, or the twenty small things your laptop happens to have installed.

A **container** pins all of it. It is a sealed box holding your code, your libraries, and a minimal operating system --- everything except the kernel itself. The same box runs identically on your laptop, on your teacher's laptop and on a server in Mumbai.

Three words you need.

An **image** is the box, built once and stored. A **container** is a running copy of an image. One image, many containers --- exactly like one class and many objects.

A **Dockerfile** is the recipe that builds the image. It is a plain text file of instructions read from top to bottom: start from this base, copy these files, run this command.

The one idea that separates people who find Docker fast from people who find it slow is **layers**. Every instruction in a Dockerfile creates a layer, and Docker caches them. If nothing above a layer changed, it is reused instantly. This is why you copy `requirements.txt` and install libraries *before* you copy your code: your code changes twenty times a day, your libraries change twice a month. Get that order wrong and every rebuild reinstalls everything.

The Indian analogy your lectures used: a container is a packed tiffin. It does not care whose kitchen it is opened in, because it brought everything with it.

Before you start

- **Docker Desktop must be installed and running before this lab.** See `labs/SETUP.md`. Check it by opening a terminal and typing `docker version` --- you should see both a Client and a Server section.
- Practical P07 is finished. You have a working FastAPI service.
- Close other heavy programs. Docker Desktop wants about 2 GB on its own.

How to run a cell

Click on a grey code cell, then press **Shift + Enter**. The cell runs and the cursor moves to the next one. A number appears in the `[ ]` on the left when the cell has finished.

Run the cells in order, from the top. A later cell almost always uses something an earlier cell created. If you jump ahead you will see a `NameError`, which just means "you have not made that thing yet".

If everything goes wrong, use the menu: **Kernel -> Restart Kernel and Clear All Outputs**, then start again from the first cell. Nothing is damaged by doing this.

In [1]:
```
# Step 0 --- check the workbench before we start.
# This cell only looks; it changes nothing. Run it and read the last line.

import sys
from pathlib import Path

print("Python  :", sys.version.split()[0])
print("Folder  :", Path.cwd().name)

_missing = []
for _name in ['numpy', 'pandas', 'sklearn', 'fastapi', 'joblib', 'requests']:
    try:
        __import__(_name)
    except ImportError:
        _missing.append(_name)

for _name in ['numpy', 'pandas', 'sklearn', 'fastapi', 'joblib', 'requests']:
    _mark = "missing" if _name in _missing else "ok"
    print(f"  {_name:<14} {_mark}")

if _missing:
    print()
    print("STOP. Some libraries are missing:", ", ".join(_missing))
    print("Ask your instructor to run the setup in labs/SETUP.md.")
else:
    print()
    print("All good. You can carry on to Step 1.")
```

```
Python  : 3.11.9
Folder  : P08-docker
```

```
numpy          ok
pandas         ok
sklearn        ok
fastapi        ok
joblib         ok
requests       ok

All good. You can carry on to Step 1.
```

Step 0b --- the dataset

Every practical in this course uses the same 600 food deliveries. The next cell makes sure the file is there.

In [2]:
```python
# The delivery-time dataset every practical in this course uses.
# If the file is missing we build it again from the same seed, so every
# student in the room gets byte-for-byte the same 600 rows.

import csv
from pathlib import Path

import numpy as np

SEED = 42
N_ROWS = 600
DATA = Path("..") / "data" / "delivery_times.csv"


def make_delivery_csv(path=DATA):
    """Write the 600-row delivery dataset. Same formula as the lectures."""
    rng = np.random.default_rng(SEED)
    distance_km = np.round(rng.uniform(0.5, 12.0, N_ROWS), 2)
    prep_time_min = np.round(rng.uniform(5, 30, N_ROWS), 0)
    traffic_level = rng.integers(1, 4, N_ROWS)
    rain = rng.binomial(1, 0.25, N_ROWS)
    delivery_min = np.round(
        6.0
        + 3.1 * distance_km
        + 0.65 * prep_time_min
        + 4.2 * traffic_level
        + 5.5 * rain
        + rng.normal(0, 2.5, N_ROWS),
        1,
    )
    path.parent.mkdir(parents=True, exist_ok=True)
    with path.open("w", newline="", encoding="utf-8") as fh:
        w = csv.writer(fh)
        w.writerow(["distance_km", "prep_time_min", "traffic_level",
                    "rain", "delivery_min"])
        for i in range(N_ROWS):
            w.writerow([distance_km[i], int(prep_time_min[i]),
                        int(traffic_level[i]), int(rain[i]), delivery_min[i]])
    return path


if not DATA.exists():
```

```
    make_delivery_csv()
    print("dataset rebuilt ->", DATA)
else:
    print("dataset found   ->", DATA)
```

```
dataset found   -> ..\data\delivery_times.csv
```

Walkthrough

Read each step, then run its cell.

Step 1 --- Check Docker is running before anything else

This lab needs Docker Desktop. Run this cell first.

If it reports that Docker is missing, the rest of the notebook will still run --- it will simply tell you which steps it skipped, rather than filling your screen with red errors. But you cannot finish the lab without it, so tell your instructor now.

In [3]:
```
import shutil
import subprocess
from pathlib import Path

WORK = Path("work")
WORK.mkdir(exist_ok=True)

IMAGE = "delivery-api:1.0"
CONTAINER = "delivery-lab"
PORT = 8010

DOCKER_OK = False
if shutil.which("docker") is None:
    print("Docker is NOT installed on this machine.")
    print("Install Docker Desktop, then restart Jupyter.")
else:
    probe = subprocess.run(["docker", "version", "--format",
                            "{{.Server.Version}}"],
                           capture_output=True, text=True)
    if probe.returncode == 0 and probe.stdout.strip():
        DOCKER_OK = True
        print("Docker server version:", probe.stdout.strip())
    else:
        print("Docker is installed but not running.")
        print("Start Docker Desktop and wait for the whale icon")
        print("to stop animating, then run this cell again.")

print()
print("DOCKER_OK =", DOCKER_OK)
```

```
Docker server version: 29.9.2

DOCKER_OK = True
```

Every later cell checks `DOCKER_OK` before running a Docker command.

Step 2 --- A helper for running Docker commands

We will run several Docker commands. This helper runs one, prints what it said, and refuses politely if Docker is not available.

In [4]:
```
def docker(*args, timeout=900, quiet=False):
    """Run one docker command and show its output."""
    if not DOCKER_OK:
        print(f"SKIPPED (no Docker): docker {' '.join(args)}")
        return None
    print(f"$ docker {' '.join(args)}")
    try:
        done = subprocess.run(["docker", *args], timeout=timeout,
                              capture_output=True, text=True,
                              encoding="utf-8", errors="replace")
    except subprocess.TimeoutExpired:
        print(f"timed out after {timeout} seconds")
        return None
    output = (done.stdout + done.stderr).strip()
    if output and not quiet:
        print(output[-2500:])
    print("exit code:", done.returncode)
    print("-" * 60)
    return done


docker("--version")
```

```
$ docker --version
Docker version 29.9.2, build a82e528
exit code: 0
------------------------------------------------------------
```

Out[4]:
```
CompletedProcess(args=['docker', '--version'], returncode=0, stdout='Docker ver
sion 29.9.2, build a82e528\n', stderr='')
```

Step 3 --- Gather what goes in the box

An image should hold exactly what the service needs, and nothing else. Our service needs three things: the trained model, the application code, and a list of libraries.

In [5]:
```
import joblib
import pandas as pd
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import train_test_split

FEATURES = ["distance_km", "prep_time_min", "traffic_level", "rain"]
orders = pd.read_csv(DATA)
X_train, _, y_train, _ = train_test_split(
    orders[FEATURES], orders["delivery_min"],
    test_size=0.2, random_state=42,
)
joblib.dump(LinearRegression().fit(X_train, y_train),
            WORK / "model.joblib")

app_py = '''"The delivery-time prediction service."
```

```
from pathlib import Path

import joblib
import pandas as pd
from fastapi import FastAPI
from pydantic import BaseModel, Field

HERE = Path(__file__).resolve().parent
FEATURES = ["distance_km", "prep_time_min", "traffic_level", "rain"]

app = FastAPI(title="Delivery Time Predictor", version="1.0.0")
model = joblib.load(HERE / "model.joblib")


class Order(BaseModel):
    distance_km: float = Field(..., gt=0, le=50)
    prep_time_min: int = Field(..., ge=0, le=120)
    traffic_level: int = Field(..., ge=1, le=3)
    rain: int = Field(..., ge=0, le=1)


@app.get("/health")
def health():
    return {"status": "ok", "model_loaded": model is not None}


@app.post("/predict")
def predict(order: Order):
    row = pd.DataFrame([order.model_dump()])[FEATURES]
    return {"delivery_min": round(float(model.predict(row)[0]), 1),
            "model_version": app.version}
'''
(WORK / "app.py").write_text(app_py, encoding="utf-8")

requirements = "\n".join([
    "fastapi==0.141.1",
    "uvicorn==0.52.1",
    "scikit-learn==1.9.0",
    "pandas==2.3.3",
    "joblib==1.5.3",
]) + "\n"
(WORK / "requirements.txt").write_text(requirements, encoding="utf-8")

for f in sorted(WORK.iterdir()):
    if f.is_file():
        print(f"  {f.name:<20} {f.stat().st_size:>9,} bytes")
```

```
 .dockerignore              72 bytes
 .dockerignore.mine         50 bytes
 app.py                    944 bytes
 Dockerfile                386 bytes
 model.joblib              921 bytes
 requirements.txt           86 bytes
```

Pinned versions again, exactly as in P01. Inside a container this matters even more: the image may be rebuilt in a year, and an unpinned `pip install` would quietly fetch different libraries.

Step 4 --- Write the Dockerfile

Read it line by line before you build it.

- `FROM` picks the starting point. `slim` is a stripped-down Debian with Python, about a fifth the size of the full image.
- `WORKDIR` is the folder inside the box where everything happens.
- `COPY requirements.txt` **before** the code --- this is the layer trick from the theory.
- `--no-cache-dir` stops pip keeping downloads we will never reuse.
- `EXPOSE` documents the port. It does not open it; `-p` does that.
- `CMD` is what runs when the container starts. `--host 0.0.0.0` is essential: the default only listens inside the box, so nothing outside could ever reach it.

In [6]:
```
dockerfile = """# The delivery-time prediction service.
FROM python:3.12-slim

WORKDIR /app

# Libraries first: this layer is cached and rarely changes.
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

# Code and model last: these change often.
COPY app.py .
COPY model.joblib .

EXPOSE 8000

CMD ["uvicorn", "app:app", "--host", "0.0.0.0", "--port", "8000"]
"""

(WORK / "Dockerfile").write_text(dockerfile, encoding="utf-8")
print((WORK / "Dockerfile").read_text(encoding="utf-8"))
```

```
# The delivery-time prediction service.
FROM python:3.12-slim

WORKDIR /app

# Libraries first: this layer is cached and rarely changes.
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

# Code and model last: these change often.
COPY app.py .
COPY model.joblib .

EXPOSE 8000

CMD ["uvicorn", "app:app", "--host", "0.0.0.0", "--port", "8000"]
```

Step 5 --- Tell Docker what to ignore

Everything in the folder is sent to Docker when you build --- that is called the **build context**. A `.dockerignore` file keeps rubbish out: caches, notebooks, virtual

environments.

Smaller context means faster builds and smaller, safer images. Never let a `.env` file or a private key into an image.

In [7]:
```
dockerignore = """__pycache__/
*.pyc
.ipynb_checkpoints/
*.ipynb
.venv/
.git/
*.db
"""

(WORK / ".dockerignore").write_text(dockerignore, encoding="utf-8")
print((WORK / ".dockerignore").read_text(encoding="utf-8"))
```

```
__pycache__/
*.pyc
.ipynb_checkpoints/
*.ipynb
.venv/
.git/
*.db
```

Step 6 --- Build the image

`-t` gives the image a name and a tag. `.` means "build using this folder".

The first build takes several minutes. It downloads the base image and installs scikit-learn and pandas, which are large. Later builds take seconds, because of the layer cache. Be patient once.

In [8]:
```
build = docker("build", "-t", IMAGE, str(WORK))
```

```
$ docker build -t delivery-api:1.0 work
```

```
#0 building with "desktop-linux" instance using docker driver

#1 [internal] load build definition from Dockerfile
#1 transferring dockerfile: 425B 0.0s done
#1 DONE 0.0s

#2 [internal] load metadata for docker.io/library/python:3.12-slim
#2 DONE 0.9s

#3 [internal] load .dockerignore
#3 transferring context: 112B done
#3 DONE 0.0s

#4 [internal] load build context
#4 transferring context: 2.07kB done
#4 DONE 0.0s

#5 [1/6] FROM docker.io/library/python:3.12-slim@sha256:b18b261e1b4974b4f321451ad
5251ded85abb22c2b8b262da527dda1d031ab30
#5 resolve docker.io/library/python:3.12-slim@sha256:b18b261e1b4974b4f321451ad525
1ded85abb22c2b8b262da527dda1d031ab30 0.0s done
#5 DONE 0.0s

#6 [2/6] WORKDIR /app
#6 CACHED

#7 [3/6] COPY requirements.txt .
#7 CACHED

#8 [4/6] RUN pip install --no-cache-dir -r requirements.txt
#8 CACHED

#9 [5/6] COPY app.py .
#9 CACHED

#10 [6/6] COPY model.joblib .
#10 CACHED

#11 exporting to image
#11 exporting layers done
#11 exporting manifest sha256:8739b2f409c2dc42e420e683d067e735b74bb62bf9cc2038acc
cffd7b896a875 done
#11 exporting config sha256:4b077beceec3ff09e29e96e449eb2aaaf6ed13c922ee95e722744
8bf615080c7 done
#11 exporting attestation manifest sha256:ff4c8192d84044748c96045c5b297f1cbe1b138
b3e262c505d9a282a17b232c6
#11 exporting attestation manifest sha256:ff4c8192d84044748c96045c5b297f1cbe1b138
b3e262c505d9a282a17b232c6 0.0s done
#11 exporting manifest list sha256:e8b5767ff097b3403cbc8052b10c2b360708d16b95a203
ed0d135a36228d032b 0.0s done
#11 naming to docker.io/library/delivery-api:1.0 done
#11 unpacking to docker.io/library/delivery-api:1.0 done
#11 DONE 0.1s
exit code: 0
----------------------------------------------------------------
```

Watch the output for the word `CACHED`. Every line that says CACHED is a layer Docker did not have to redo.

Step 7 --- Look at what you built

Images are stored by Docker, not in your folder. Ask it what it has.

In [9]:
```
docker("images", IMAGE)
```

```
$ docker images delivery-api:1.0
IMAGE              ID             DISK USAGE   CONTENT SIZE   EXTRA
delivery-api:1.0   e8b5767ff097        680MB          156MB
exit code: 0
------------------------------------------------------------
```

Out[9]:
```
CompletedProcess(args=['docker', 'images', 'delivery-api:1.0'], returncode=0, s
tdout='IMAGE              ID             DISK USAGE   CONTENT SIZE   EXTRA\ndel
ivery-api:1.0   e8b5767ff097        680MB          156MB        \n', stderr='')
```

Expect roughly 500-700 MB. Most of that is scikit-learn, pandas and numpy, not your code. If you had started `FROM python:3.12` instead of `python:3.12-slim` it would be about 300 MB larger for no gain.

Step 8 --- Run it

`-d` runs it in the background. `--rm` deletes the container when it stops, so your machine stays tidy. `-p 8010:8000` connects port 8010 on your laptop to port 8000 inside the box.

Read that port mapping carefully. It is the single most common source of confusion in this lab.

In [10]:
```
import time

docker("rm", "-f", CONTAINER, quiet=True)      # clear any leftover
run = docker("run", "-d", "--rm", "--name", CONTAINER,
             "-p", f"{PORT}:8000", IMAGE)

if DOCKER_OK:
    time.sleep(6)          # give uvicorn a moment to start
    docker("ps", "--filter", f"name={CONTAINER}")
```

```
$ docker rm -f delivery-lab
exit code: 0
------------------------------------------------------------
$ docker run -d --rm --name delivery-lab -p 8010:8000 delivery-api:1.0
7f9d3f26ff4811a70a4173a30429adcaad408280cf5d392107055838316b574a
exit code: 0
------------------------------------------------------------
$ docker ps --filter name=delivery-lab
CONTAINER ID   IMAGE              COMMAND                  CREATED         STATUS
PORTS                                       NAMES
7f9d3f26ff48   delivery-api:1.0   "uvicorn app:app --h…"   6 seconds ago   Up 6 s
econds   0.0.0.0:8010->8000/tcp, [::]:8010->8000/tcp   delivery-lab
exit code: 0
------------------------------------------------------------
```

Step 9 --- Talk to the service inside the container

Your service is now running in a sealed box, reachable on port 8010. Nothing about this code knows or cares that it is in a container.

In [11]:
```
import requests

if DOCKER_OK:
    base = f"http://127.0.0.1:{PORT}"
    try:
        health = requests.get(f"{base}/health", timeout=10)
        print("GET /health ->", health.status_code, health.json())

        order = {"distance_km": 5.0, "prep_time_min": 20,
                 "traffic_level": 2, "rain": 0}
        pred = requests.post(f"{base}/predict", json=order,
                             timeout=10)
        print("POST /predict ->", pred.status_code, pred.json())
    except requests.RequestException as problem:
        print("Could not reach the container:", problem)
        print("Run the previous cell again, wait, then retry.")
else:
    print("SKIPPED (no Docker): nothing is listening on port",
          PORT)
```

```
GET /health -> 200 {'status': 'ok', 'model_loaded': True}
POST /predict -> 200 {'delivery_min': 43.0, 'model_version': '1.0.0'}
```

That prediction came from a model inside a container. Copy that image to any machine with Docker and you get the same answer.

Step 10 --- Read its logs, then stop it

A container writes its output to a log Docker keeps. `docker logs` is how you read it --- there is no log file to open.

Always stop what you started. A forgotten container holds a port and memory.

In [12]:
```
docker("logs", "--tail", "15", CONTAINER)
docker("stop", CONTAINER)

print()
print("Containers still running:")
docker("ps")
```

```
$ docker logs --tail 15 delivery-lab
INFO:     172.17.0.1:56398 - "GET /health HTTP/1.1" 200 OK
INFO:     172.17.0.1:56410 - "POST /predict HTTP/1.1" 200 OK
INFO:     Started server process [1]
INFO:     Waiting for application startup.
INFO:     Application startup complete.
INFO:     Uvicorn running on http://0.0.0.0:8000 (Press CTRL+C to quit)
exit code: 0
------------------------------------------------------------
$ docker stop delivery-lab
```

```
delivery-lab
exit code: 0
------------------------------------------------------------

Containers still running:
$ docker ps
CONTAINER ID   IMAGE     COMMAND   CREATED   STATUS    PORTS     NAMES
exit code: 0
------------------------------------------------------------
```

Out[12]:
```
CompletedProcess(args=['docker', 'ps'], returncode=0, stdout='CONTAINER ID   IM
AGE     COMMAND   CREATED   STATUS    PORTS     NAMES\n', stderr='')
```

Notice the service logged each request. In P10 you will make those log lines worth searching, and in P09 a cluster will read this same output.

Your turn

The walkthrough above is finished. Now you write some code.

There are **3 tasks**. Each one is small. Each one has a hint. Do them in order.

Where you see `# TODO`, replace that line with your own code. Do not delete the variable name on the left of the `=` sign --- the self-check at the bottom looks for exactly that name.

When you have tried all three, run the **self-check** cell at the end. It prints a table telling you which tasks are correct. You can run it as many times as you like.

Task T1 --- Keep the rubbish out of your image

Write a **new** file `work/.dockerignore.mine` that excludes all of the following, one pattern per line:

- `__pycache__/`
- `*.pyc`
- `.venv/`
- `.git/`
- `tests/`
- `*.md`

Store the list of lines you wrote in `T1_patterns`.

> **Hint.** Make a Python list of the six strings, join them with `"\n"`, and write the result with `(WORK / ".dockerignore.mine").write_text(...)`. Set `T1_patterns` to the same list.

In [13]:
```
# the six patterns, in a list
T1_patterns = ["__pycache__/", "*.pyc", ".venv/", ".git/", "tests/", "*.md"]
```

```
# write them to work/.dockerignore.mine, one per line
text = "\n".join(T1_patterns) + "\n"
(WORK / ".dockerignore.mine").write_text(text, encoding="utf-8")

print(T1_patterns)
```

```
['__pycache__/', '*.pyc', '.venv/', '.git/', 'tests/', '*.md']
```

Task T2 --- Fix a broken Dockerfile

Below is a Dockerfile with **three** faults. Find them and write a
corrected version into `T2_fixed` (a string).

The three faults are:

1. The code is copied **before** the requirements are installed, so
   every rebuild reinstalls every library.
2. `uvicorn` is started without `--host 0.0.0.0`, so nothing outside the
   container can ever reach it.
3. It copies the whole folder with `COPY . .` *and* has no
   `WORKDIR`, so files land in the image root.

Your corrected version must have a `WORKDIR`, must copy
`requirements.txt` and run `pip install` before copying `app.py`, and
its `CMD` must contain `--host` and `0.0.0.0`.

> **Hint.** Do not try to patch the broken text. Write the corrected Dockerfile out fresh --- Step 4 has a working one you can adapt. Assign the whole thing to `T2_fixed` as a triple-quoted string.

In [14]:
```
T2_broken = '''FROM python:3.12-slim
COPY . .
RUN pip install --no-cache-dir -r requirements.txt
EXPOSE 8000
CMD ["uvicorn", "app:app", "--port", "8000"]
'''

print("THE BROKEN ONE:")
print(T2_broken)

# corrected Dockerfile:
# 1. WORKDIR /app, so files do not land in the image root
# 2. requirements are installed BEFORE the code is copied, so the layer stays ca
# 3. --host 0.0.0.0, so the service can be reached from outside the container
T2_fixed = '''FROM python:3.12-slim

WORKDIR /app

COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

COPY app.py .
```

```
COPY model.joblib .

EXPOSE 8000

CMD ["uvicorn", "app:app", "--host", "0.0.0.0", "--port", "8000"]
'''

print("YOURS:")
print(T2_fixed)
```

```
THE BROKEN ONE:
FROM python:3.12-slim
COPY . .
RUN pip install --no-cache-dir -r requirements.txt
EXPOSE 8000
CMD ["uvicorn", "app:app", "--port", "8000"]

YOURS:
FROM python:3.12-slim

WORKDIR /app

COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

COPY app.py .
COPY model.joblib .

EXPOSE 8000

CMD ["uvicorn", "app:app", "--host", "0.0.0.0", "--port", "8000"]
```

Task T3 --- Rebuild and serve a rainy order

This task needs Docker running.

Build your corrected Dockerfile from T2 as a second image tagged `delivery-api:2.0`, run it on port **8011**, and ask it to predict a 4 km order, 30 minutes prep, traffic level 3, in the rain.

Store the predicted minutes in `T3_minutes`, then stop the container.

> **Hint.** Write `T2_fixed` to `work/Dockerfile.v2`, then build with `docker("build", "-f", str(WORK / "Dockerfile.v2"), "-t", "delivery-api:2.0", str(WORK))`. Run it exactly as Step 8 did but with a different name and port, sleep a few seconds, then POST to `http://127.0.0.1:8011/predict`. The build is quick because the layers are already cached.

In [15]:
```
T3_minutes = None

if not DOCKER_OK:
    print("SKIPPED: this task needs Docker Desktop running.")
else:
    # write the corrected Dockerfile to work/Dockerfile.v2
```

```
    (WORK / "Dockerfile.v2").write_text(T2_fixed, encoding="utf-8")

    # build it as delivery-api:2.0 (quick, because the layers are cached)
    docker("build", "-f", str(WORK / "Dockerfile.v2"), "-t", "delivery-api:2.0",

    # run it as container "delivery-lab-2" on port 8011
    docker("rm", "-f", "delivery-lab-2", quiet=True)
    docker("run", "-d", "--rm", "--name", "delivery-lab-2", "-p", "8011:8000", "
    time.sleep(6)          # give uvicorn a moment to start

    # POST the rainy order and read delivery_min
    rainy_order = {"distance_km": 4.0, "prep_time_min": 30,
                   "traffic_level": 3, "rain": 1}
    reply = requests.post("http://127.0.0.1:8011/predict", json=rainy_order, tim
    print("POST /predict ->", reply.status_code, reply.json())
    T3_minutes = reply.json()["delivery_min"]

    # stop the container
    docker("stop", "delivery-lab-2")

print("T3_minutes =", T3_minutes)
```

```
$ docker build -f work\Dockerfile.v2 -t delivery-api:2.0 work
```

```
#0 building with "desktop-linux" instance using docker driver

#1 [internal] load build definition from Dockerfile.v2
#1 transferring dockerfile: 282B done
#1 DONE 0.0s

#2 [internal] load metadata for docker.io/library/python:3.12-slim
#2 DONE 0.4s

#3 [internal] load .dockerignore
#3 transferring context: 112B done
#3 DONE 0.0s

#4 [internal] load build context
#4 transferring context: 95B done
#4 DONE 0.0s

#5 [1/6] FROM docker.io/library/python:3.12-slim@sha256:b18b261e1b4974b4f321451ad
5251ded85abb22c2b8b262da527dda1d031ab30
#5 resolve docker.io/library/python:3.12-slim@sha256:b18b261e1b4974b4f321451ad525
1ded85abb22c2b8b262da527dda1d031ab30 0.0s done
#5 DONE 0.0s

#6 [4/6] RUN pip install --no-cache-dir -r requirements.txt
#6 CACHED

#7 [5/6] COPY app.py .
#7 CACHED

#8 [2/6] WORKDIR /app
#8 CACHED

#9 [3/6] COPY requirements.txt .
#9 CACHED

#10 [6/6] COPY model.joblib .
#10 CACHED

#11 exporting to image
#11 exporting layers done
#11 exporting manifest sha256:8739b2f409c2dc42e420e683d067e735b74bb62bf9cc2038acc
cffd7b896a875 done
#11 exporting config sha256:4b077beceec3ff09e29e96e449eb2aaaf6ed13c922ee95e722744
8bf615080c7 done
#11 exporting attestation manifest sha256:0d3e6818105adb7b0e69f73558b7e7d97cc7e25
b45ec46d2bd768216dc8ac690 0.0s done
#11 exporting manifest list sha256:40b2436523e333d3a7e1fa9aac5eaaba891fcfbad64c82
61aa87a10a940c4b70
#11 exporting manifest list sha256:40b2436523e333d3a7e1fa9aac5eaaba891fcfbad64c82
61aa87a10a940c4b70 0.0s done
#11 naming to docker.io/library/delivery-api:2.0 done
#11 unpacking to docker.io/library/delivery-api:2.0 done
#11 DONE 0.1s
exit code: 0
------------------------------------------------------------
$ docker rm -f delivery-lab-2
exit code: 0
------------------------------------------------------------
$ docker run -d --rm --name delivery-lab-2 -p 8011:8000 delivery-api:2.0
```

```
3cea596248788bdbba44207ed4c559b60c86c1266c5ca4bd1fec77c90f58adf3
exit code: 0
------------------------------------------------------------
POST /predict -> 200 {'delivery_min': 56.1, 'model_version': '1.0.0'}
$ docker stop delivery-lab-2
delivery-lab-2
exit code: 0
------------------------------------------------------------
T3_minutes = 56.1
```

Self-check

Run the cell below to mark your work.

In [16]:
```
# ------------------------------------------------------------------
# SELF-CHECK --- run this when you have attempted the tasks above.
# It never breaks your notebook. A task you have not done yet simply
# shows FAIL.
# ------------------------------------------------------------------

_results = []


def _check(label, fn):
    """Evaluate one graded condition without ever raising."""
    try:
        ok = bool(fn())
    except Exception:
        ok = False
    _results.append((label, ok))


_check('T1 | the file exists', lambda: (WORK / '.dockerignore.mine').is_file())
_check('T1 | it contains all six patterns, one per line', lambda: set(ln.strip()
_check('T1 | T1_patterns lists the same six', lambda: set(T1_patterns) == {'__py
_check('T2 | the fixed Dockerfile sets a WORKDIR', lambda: 'WORKDIR' in T2_fixed
_check('T2 | requirements are installed before the code is copied', lambda: T2_f
_check('T2 | uvicorn is told to listen on 0.0.0.0', lambda: '--host' in T2_fixed
_check('T2 | it still starts from a slim python base and exposes a port', lambda
_check('T3 | the second Dockerfile was written', lambda: (WORK / 'Dockerfile.v2'
_check('T3 | the container returned a sensible prediction', lambda: 30 < float(T

print("================================================================")
print("SELF-CHECK   Practical 08 --- Packing the Service into a Container")
print("================================================================")
for _label, _ok in _results:
    print(f"  [{'PASS' if _ok else 'FAIL'}]  {_label}")
print("----------------------------------------------------------------")
_passed = sum(1 for _, _ok in _results if _ok)
print(f"  {_passed} of {len(_results)} checks passed")
print("================================================================")
if _passed == len(_results):
    print("Well done. Save the notebook and submit it.")
else:
    print("Read the FAIL lines above, fix those tasks, run this cell again.")
```

```
================================================================
SELF-CHECK   Practical 08 --- Packing the Service into a Container
================================================================
  [PASS]  T1 | the file exists
  [PASS]  T1 | it contains all six patterns, one per line
  [PASS]  T1 | T1_patterns lists the same six
  [PASS]  T2 | the fixed Dockerfile sets a WORKDIR
  [PASS]  T2 | requirements are installed before the code is copied
  [PASS]  T2 | uvicorn is told to listen on 0.0.0.0
  [PASS]  T2 | it still starts from a slim python base and exposes a port
  [PASS]  T3 | the second Dockerfile was written
  [PASS]  T3 | the container returned a sensible prediction
----------------------------------------------------------------
  9 of 9 checks passed
================================================================
Well done. Save the notebook and submit it.
```

What to submit

1. This notebook, with every cell run and its output visible.
2. Your `work/Dockerfile` and `work/.dockerignore`.
3. A screenshot of the output of `docker images delivery-api:1.0` showing the size.

Name your file `PXX_<your-roll-number>.ipynb` before you upload it.

How this practical is marked

What is marked	Marks
Walkthrough run end to end, image built and container answered	3
Task T1 --- a correct .dockerignore	2
Task T2 --- the broken Dockerfile diagnosed and fixed	3
Task T3 --- a rebuilt image serving a prediction	2
Total	**10**

Read more

- Docker --- Dockerfile reference --- https://docs.docker.com/reference/dockerfile/
- Docker --- Building best practices --- https://docs.docker.com/build/building/best-practices/
- Docker --- .dockerignore file --- https://docs.docker.com/build/concepts/context/#dockerignore-files
- Docker Hub --- the official python image --- https://hub.docker.com/_/python